Algorithms and Development of Programmes for Biological Chemistry

Agata M. Kilar

Research Group Bioinformatics and Computational Biology
Department for Theoretical Chemistry
Währingerstrasse 17, 1090 Vienna, Austria

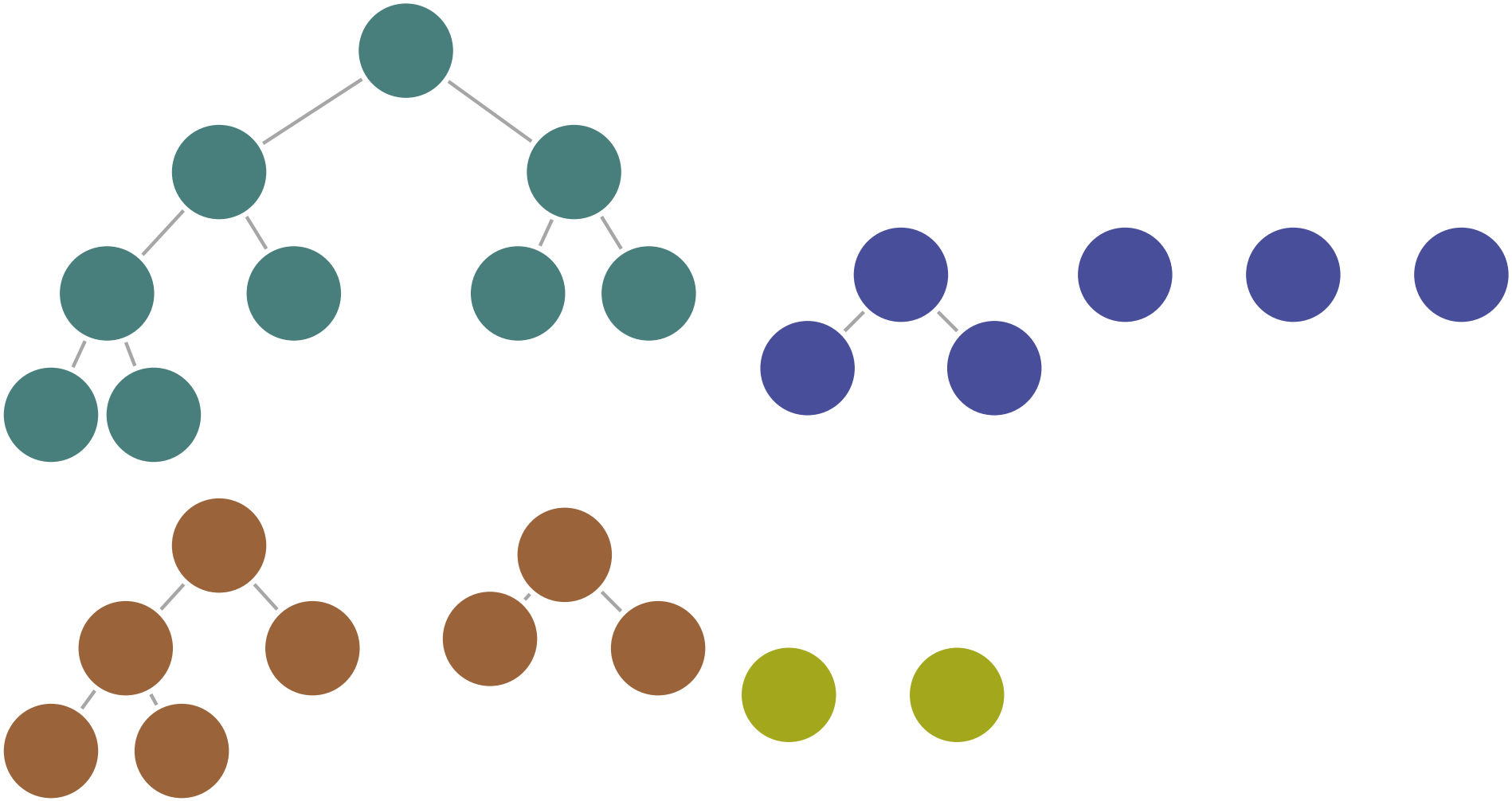
Summer 2026

Introduction to Dynamic Programming

- **Correctness** of **brute force** and **efficiency** of **greedy** algorithms
- Two common use-cases are:
  1. finding an **optimal solution**
  2. **counting** the total number of **solutions** for a problem

- Invented by Richard Bellman in the 1950s
- Misleading name - cover up for "research" but sounds complicated

Dynamic Programming is about identifying and solving all subproblems



Fibonacci Series

N-th Fibonacci number

Write a function that returns the n-th Fibonacci number.

$$F_1 = F_2 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

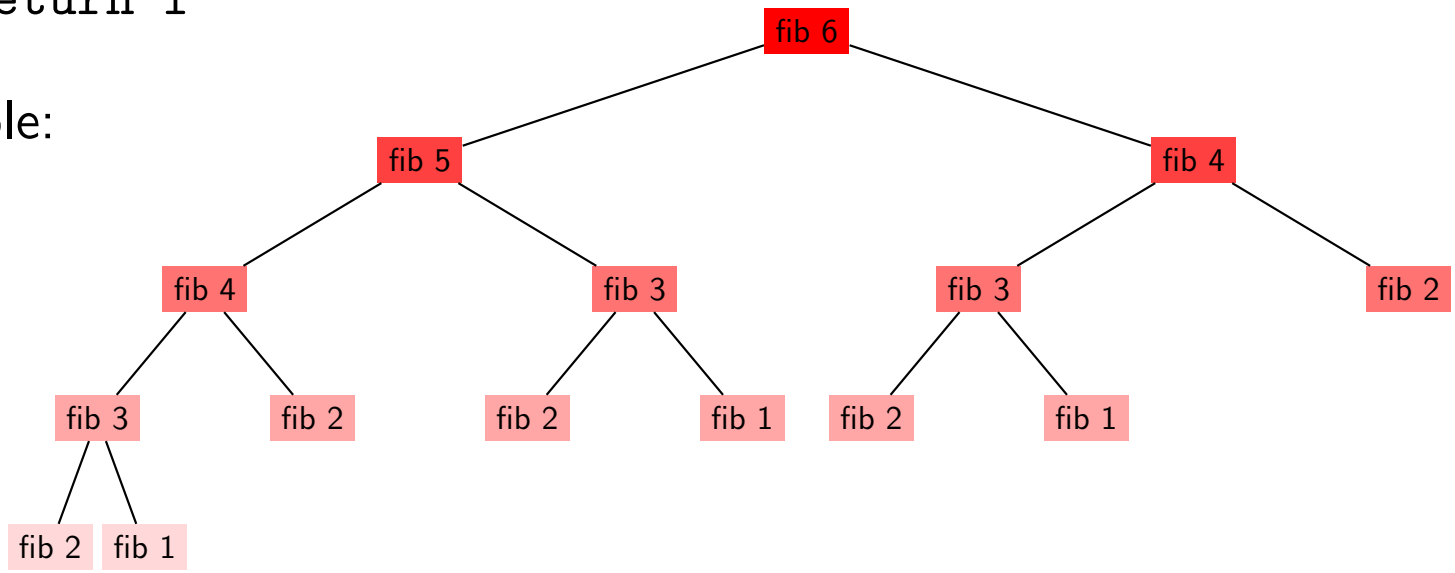
n	1	2	3	4	5	6	7
F_n	1	1	2	3	5	8	13

Fibonacci Series

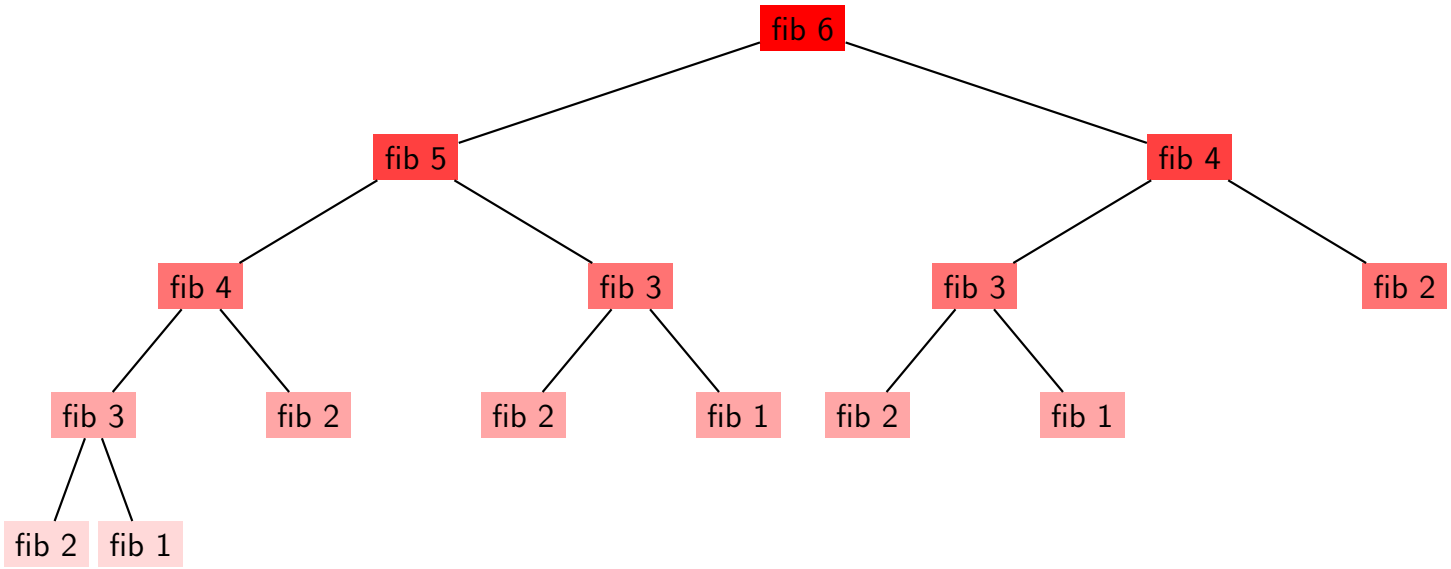
The Fibonacci Series of numbers 1, 1, 2, 3, 5, 8, 13, 21, 34, ... can be defined recursively:

```
def fib(n):
    if n <= 2:  f = 1
    else:       f = fib(n-1) + fib(n-2)
    return f
```

Example:

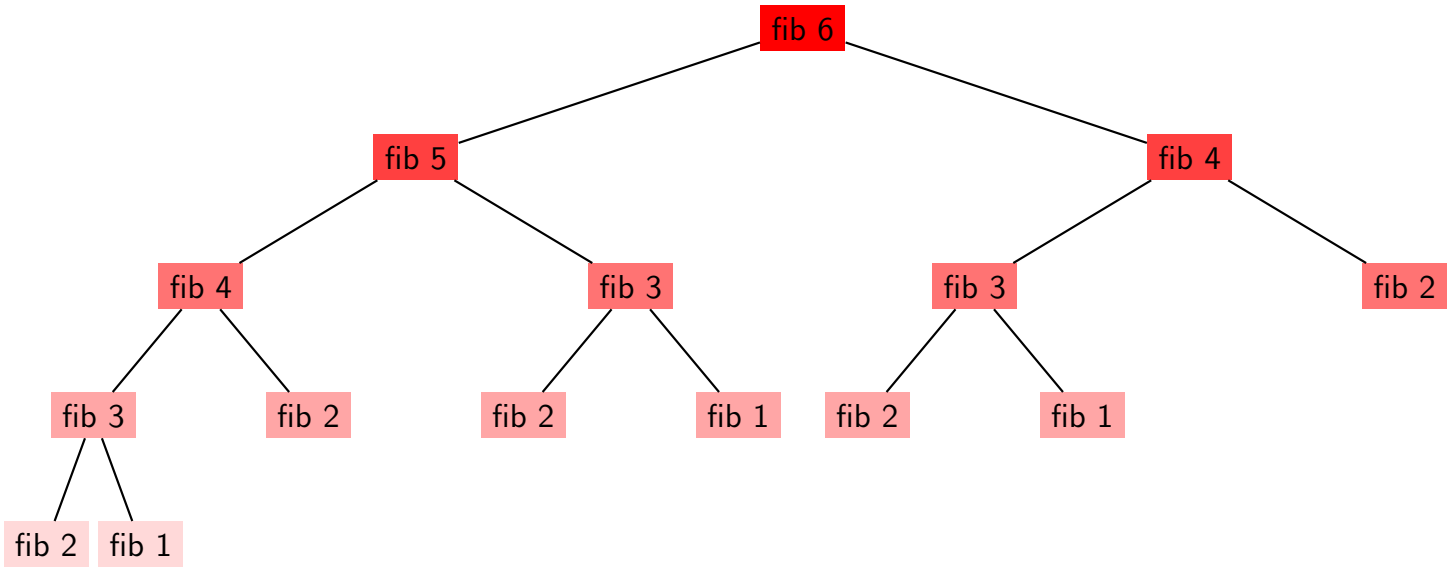


DEMO



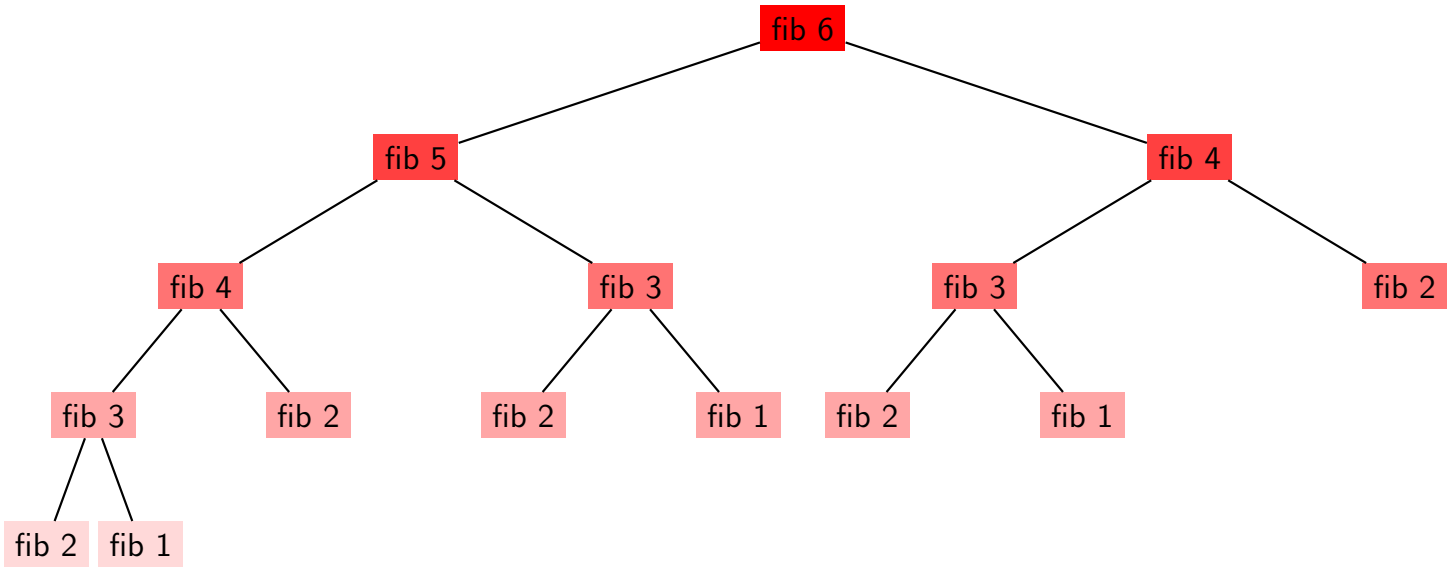
Building the solution space we realize:

1. The run-time of the recursion scales exponentially ($\sim 2^n$) with the input size.
2. Many (sub)solutions are calculated over and over again ($5 \times$ `fib 2`, $3 \times$ `fib 3`, $2 \times$ `fib 4`).



Building the solution space we realize:

1. The run-time of the recursion scales exponentially ($\sim 2^n$) with the input size.
2. Many (sub)solutions are calculated over and over again ($5 \times$ `fib 2`, $3 \times$ `fib 3`, $2 \times$ `fib 4`).
3. Avoid redundancy: tabulate subsolutions!



Building the solution space we realize:

1. The run-time of the recursion scales exponentially ($\sim 2^n$) with the input size.
2. Many (sub)solutions are calculated over and over again ($5 \times$ `fib 2`, $3 \times$ `fib 3`, $2 \times$ `fib 4`).
3. Avoid redundancy: tabulate subsolutions!

Dynamic Programming = Recursion + Memoization

Dynamic Programming = Recursion + Memoization

```
def fib(n):
    if n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    return f
```

```
memo = {}
def fib(n):
    if n in memo:
        return memo[n]
    elif n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    memo[n] = f
    return f
```

Dynamic Programming = Recursion + Memoization

```
def fib(n):
    if n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    return f
```

```
memo = {}
def fib(n):
    if n in memo:
        return memo[n]
    elif n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    memo[n] = f
    return f
```

Memoization - remember each computation of, e.g., a Fibonacci number, and compute it at most once.

- First, do a dictionary lookup
- Compute the value only if it is not in the dictionary

You can transform any recursive algorithm into a DP algorithm by adding memoization!

Dynamic Programming = Recursion + Memoization

```
def fib(n):
    if n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    return f
```

```
memo = {}
def fib(n):
    if n in memo:
        return memo[n]
    elif n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    memo[n] = f
    return f
```

Stop and think:

- How much faster can one compute fib using DP?
- How much space do we need for DP-fib?

Recursive (top-down) vs. iterative (bottom-up) Fibonacci

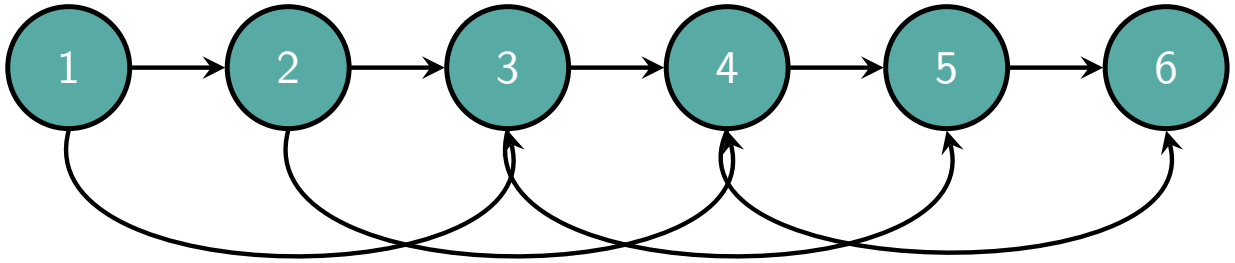
```
memo = {}
def fib(n):
    if n in memo:
        return memo[n]
    elif n <= 2:
        f = 1
    else:
        f = fib(n-1)
            + fib(n-2)
    memo[n] = f
    return f
```

```
memo = {}
def fib(n):

    for i in range(1,n+1):
        if i <= 2:
            f = 1
        else:
            f = memo[i-1]
                + memo[i-2]
        memo[i] = f
    return memo[n]
```

DP = Order + Tabulation (Reuse)

Iterative DP only works once we know a valid dependency order



Fibonacci DAG for Fibonacci subproblems is very simple: each node n depends on $n-1$ and $n-2$.

Dependencies graph can't form a cycle!

DEMO

Optimization by DP: Levenshtein Distance

Optimization by DP: Levenshtein Distance

Definition: *Levenshtein distance of strings a and b* := minimal cost of transforming *a* into *b* by edit operations "replace", "insert", "delete" each of cost 1.

Example: `AUTO` $\Rightarrow$ `RAD`, `MOTORRAD` $\Rightarrow$ `FAHRRAD`

As edit sequence:

`AUTO` $\rightarrow$ `AUT` $\rightarrow$ `AUD` $\rightarrow$ `AD` $\rightarrow$ `RAD` (4)

`MOTORRAD` $\rightarrow$ `MOTHRRAD` $\rightarrow$ `MOAHRRAD` $\rightarrow$ `MFAHRRAD` $\rightarrow$ `FAHRRAD` (4)

or as alignment:

```
-AUTO      MOTORRAD
RA-D-      -FAHRRAD
```

Optimization by DP: Levenshtein Distance

Definition: *Levenshtein distance of strings a and b* := minimal cost of transforming *a* into *b* by edit operations "replace", "insert", "delete" each of cost 1.

Example: `AUTO` $\Rightarrow$ `RAD`, `MOTORRAD` $\Rightarrow$ `FAHRRAD`

As edit sequence:

`AUTO` $\rightarrow$ `AUT` $\rightarrow$ `AUD` $\rightarrow$ `AD` $\rightarrow$ `RAD` (4)

`MOTORRAD` $\rightarrow$ `MOTHRRAD` $\rightarrow$ `MOAHRRAD` $\rightarrow$ `MFAHRRAD` $\rightarrow$ `FAHRRAD` (4)

or as alignment:

```
-AUTO        MOTORRAD
RA-D-        -FAHRRAD
```

NOTE: "Levenshtein Distance" has *optimal substructure*:
If `-AUTO` is optimal, then it's subsolution `-AUT` (of subproblem
`RA-D-` `RA-D`
'AUT' vs. 'RAD') must be optimal.

Calculating the Levenshtein distance of a and b

- build $n \times m$-matrix D
  where: $D_{ij} :=$ distance of $a_1, \ldots, a_i$ and $b_1, \ldots, b_j$
- calculate each D_{ij} from optimal partial solutions

Recursion:

$$D_{0,0} = 0; D_{0,j} = j; D_{i,0} = i$$

$$D_{i,j} = min \begin{cases} D_{i-1,j-1} + \begin{cases} 1 & \text{if } a_i \neq b_j \\ 0 & \text{otherwise} \end{cases} \\ D_{i-1,j} + 1 \\ D_{i,j-1} + 1 \end{cases}$$

Recursion:

$$D_{0,0} = 0; D_{0,j} = j; D_{i,0} = i$$

$$D_{i,j} = min \begin{cases} D_{i-1,j-1} + \begin{cases} 1 & \text{if } a_i \neq b_j \\ 0 & \text{otherwise} \end{cases} \\ D_{i-1,j} + 1 \\ D_{i,j-1} + 1 \end{cases}$$

	j	0	1	2	3
i		-	R	A	D
0	-	0	1	2	3
1	A	1			
2	U	2			
3	T	3			
4	O	4			

Recursion:

$$D_{0,0} = 0; D_{0,j} = j; D_{i,0} = i$$

$$D_{i,j} = min \begin{cases} D_{i-1,j-1} + \begin{cases} 1 & \text{if } a_i \neq b_j \\ 0 & \text{otherwise} \end{cases} \\ D_{i-1,j} + 1 \\ D_{i,j-1} + 1 \end{cases}$$

	j	0	1	2	3
i		-	R	A	D
0	-	0	1	2	3
1	A	1			
2	U	2			
3	T	3			
4	O	4			

	j	0	1	2	3
i		-	R	A	D
0	-	**0**	**1**	2	3
1	A	1	1	**1**	2
2	U	2	2	2	**2**
3	T	3	3	3	**3**
4	O	4	4	4	**4**

Recursion:

$$D_{0,0} = 0; D_{0,j} = j; D_{i,0} = i$$

$$D_{i,j} = min \begin{cases} D_{i-1,j-1} + \begin{cases} 1 & \text{if } a_i \neq b_j \\ 0 & \text{otherwise} \end{cases} \\ D_{i-1,j} + 1 \\ D_{i,j-1} + 1 \end{cases}$$

	j	0	1	2	3
i		-	R	A	D
0	-	0	1	2	3
1	A	1			
2	U	2			
3	T	3			
4	O	4			

	j	0	1	2	3
i		-	R	A	D
0	-	**0**	**1**	2	3
1	A	1	1	**1**	2
2	U	2	2	2	**2**
3	T	3	3	3	**3**
4	O	4	4	4	**4**

$D_{n,m}$ contains the distance of a and b.

Tracing back the optimal choices from $D_{n,m}$ to $D_{0,0}$ yields *some* optimum alignment of a and b.

From minimizing cost to maximing score

DNA sequence matching

DNA sequence matching

Goal: compare two DNA sequences and find a best alignment.

Example:

```
A C T G A T T C A
A C G C A T C A
```

We use the scoring system:

- gap penalty: -2
- mismatch penalty: -3
- match bonus: $+2$

Task: among all possible alignments, find one with the best score.

DNA sequence matching: matrix

$a = \texttt{ACTGATTCA}, \qquad b = \texttt{ACGCATCA}$

		-	A	C	G	C	A	T	C	A
	-	0	-2	-4	-6	-8	-10	-12	-14	-16
1	A	-2								
2	C	-4								
3	T	-6								
4	G	-8								
5	A	-10								
6	T	-12								
7	T	-14								
8	C	-16								
9	A	-18								**X**

$D_{9,8} = X$ is the score of the best global alignment.

DNA sequence matching: solved matrix

$a = \texttt{ACTGATTCA}, \qquad b = \texttt{ACGCATCA}$

		-	A	C	G	C	A	T	C	A
	-	0	-2	-4	-6	-8	-10	-12	-14	-16
1	A	-2	2	0	-2	-4	-6	-8	-10	-12
2	C	-4	0	4	2	0	-2	-4	-6	-8
3	T	-6	-2	2	1	-1	-3	0	-2	-4
4	G	-8	-4	0	4	2	0	-2	-3	-5
5	A	-10	-6	-2	2	1	4	2	0	-1
6	T	-12	-8	-4	0	-1	2	6	4	2
7	T	-14	-10	-6	-2	-3	0	4	3	1
8	C	-16	-12	-8	-4	0	-2	2	6	4
9	A	-18	-14	-10	-6	-2	2	0	4	**8**

$D_{9,8} = 8$ is the score of the best global alignment.

DNA sequence matching: optimal alignment

One optimal alignment is

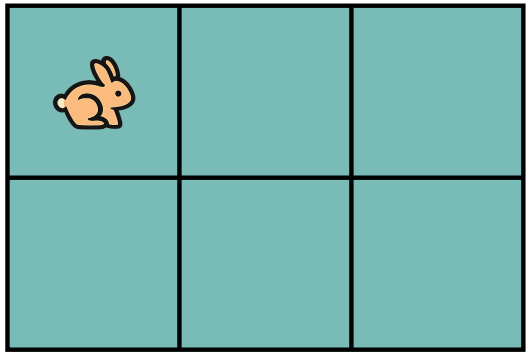
```
ACTG-ATTCA
AC-GCAT-CA
```

		-	A	C	G	C	A	T	C	A
	-	**0**	-2	-4	-6	-8	-10	-12	-14	-16
1	A	-2	**2**	0	-2	-4	-6	-8	-10	-12
2	C	-4	0	**4**	2	0	-2	-4	-6	-8
3	T	-6	-2	**2**	1	-1	-3	0	-2	-4
4	G	-8	-4	0	**4**	**2**	0	-2	-3	-5
5	A	-10	-6	-2	2	1	**4**	2	0	-1
6	T	-12	-8	-4	0	-1	2	**6**	4	2
7	T	-14	-10	-6	-2	-3	0	**4**	3	1
8	C	-16	-12	-8	-4	0	-2	2	**6**	4
9	A	-18	-14	-10	-6	-2	2	0	4	**8**

Maze problem

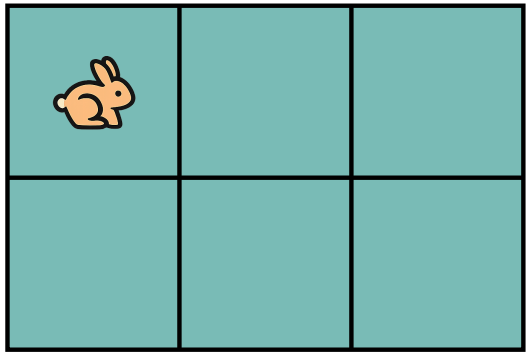
Maze problem

Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



Maze problem

Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



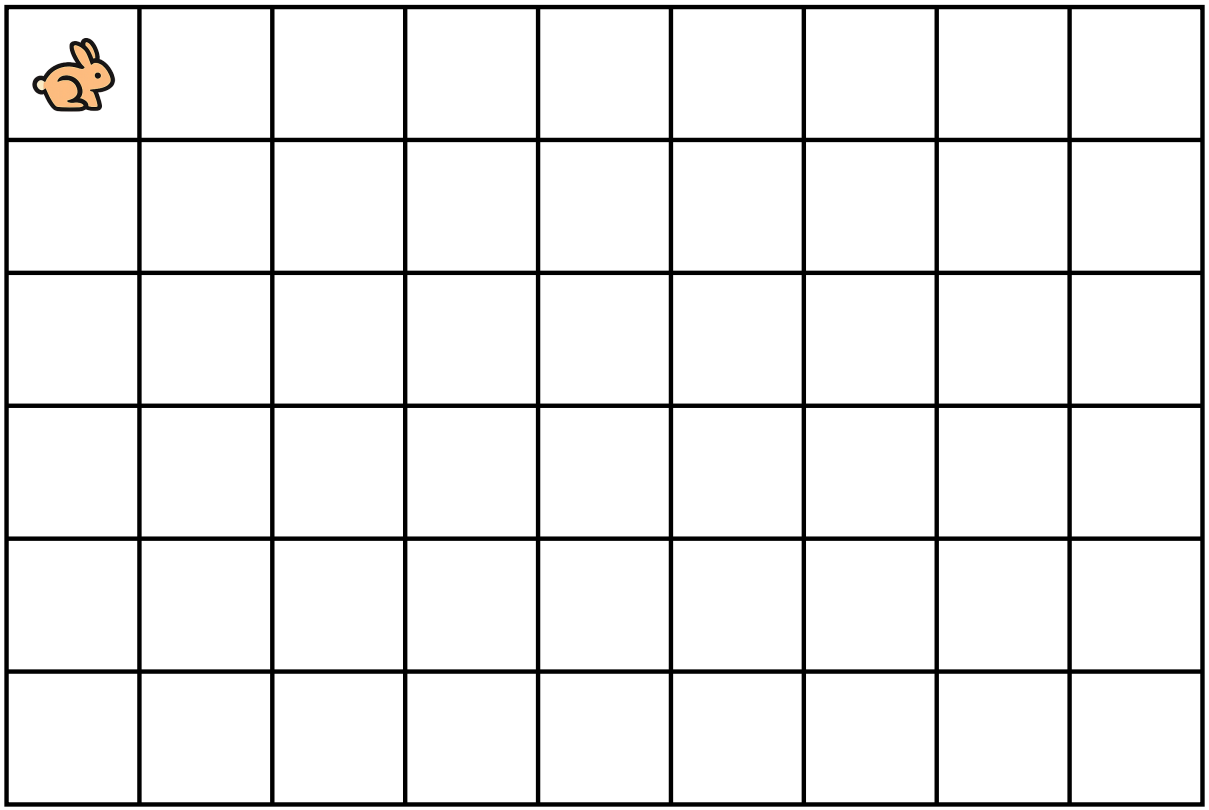
Right > Right > Down

Right > Down > Right

Down > Right > Right

Maze problem

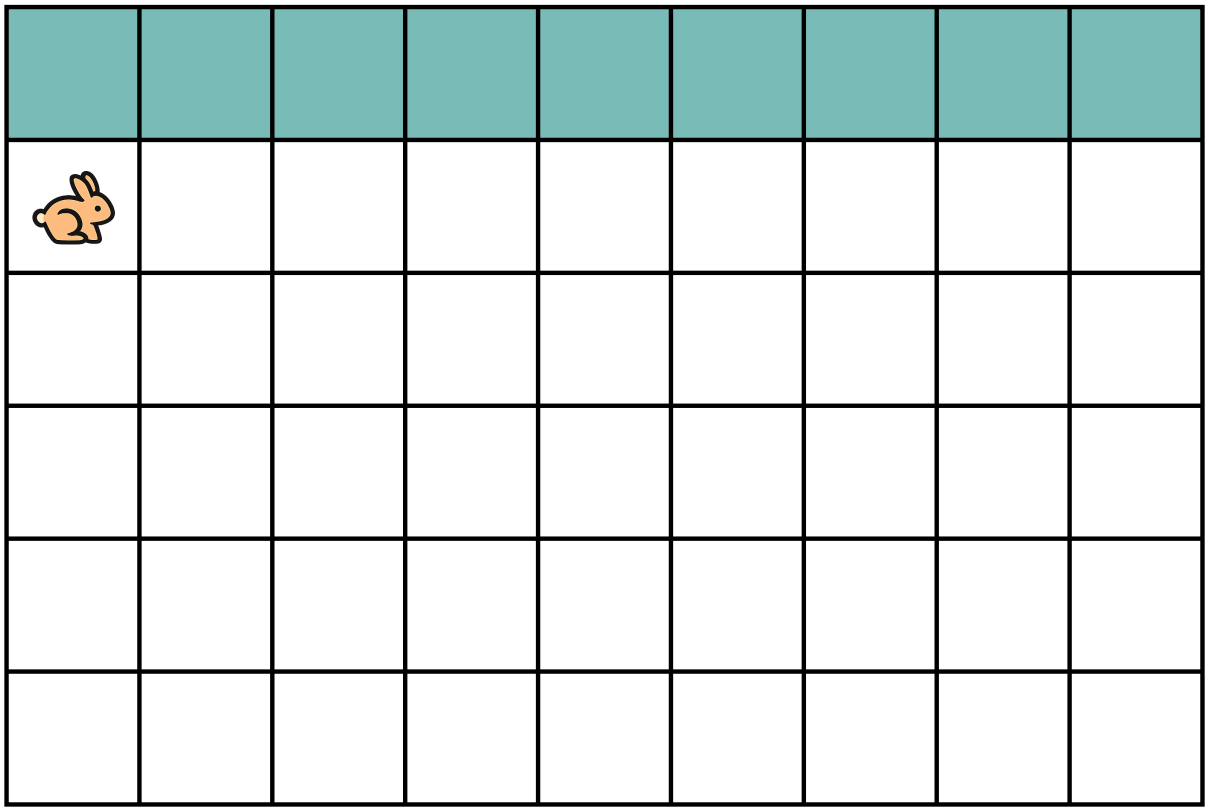
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



6×9

Maze problem

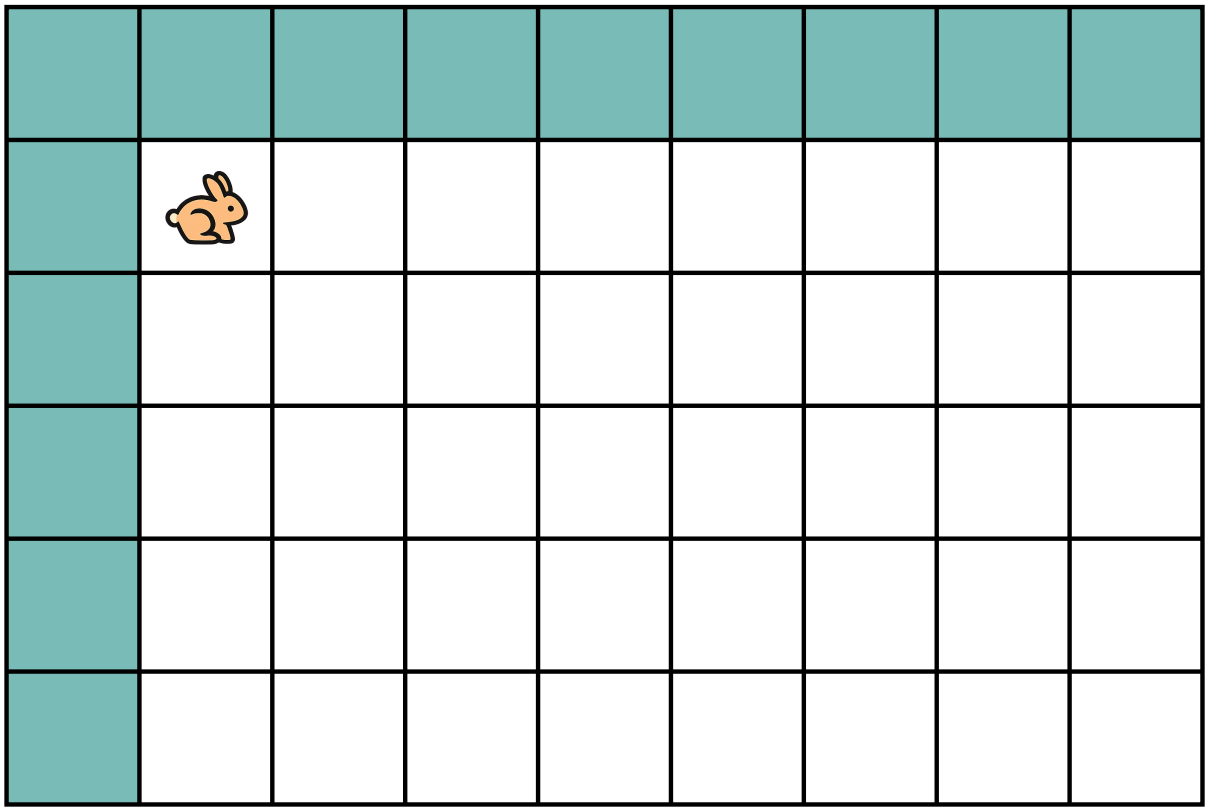
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



5×9

Maze problem

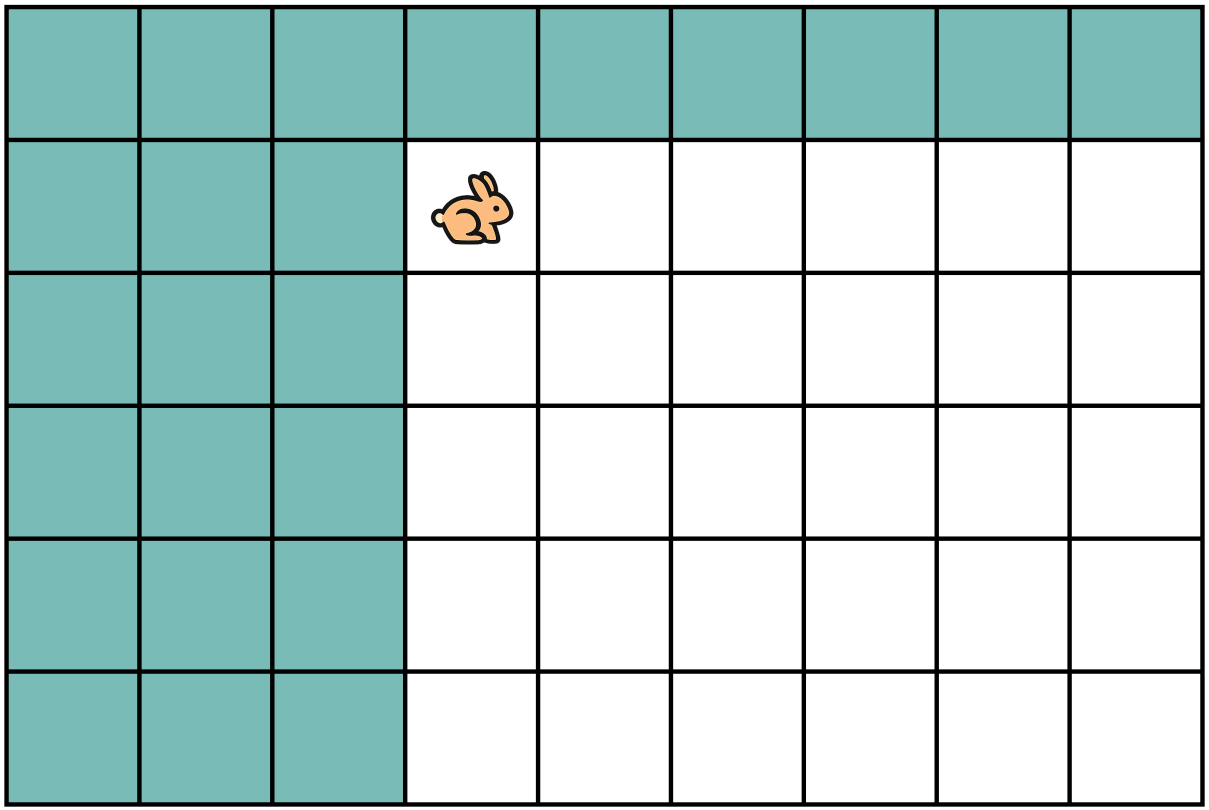
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



5×8

Maze problem

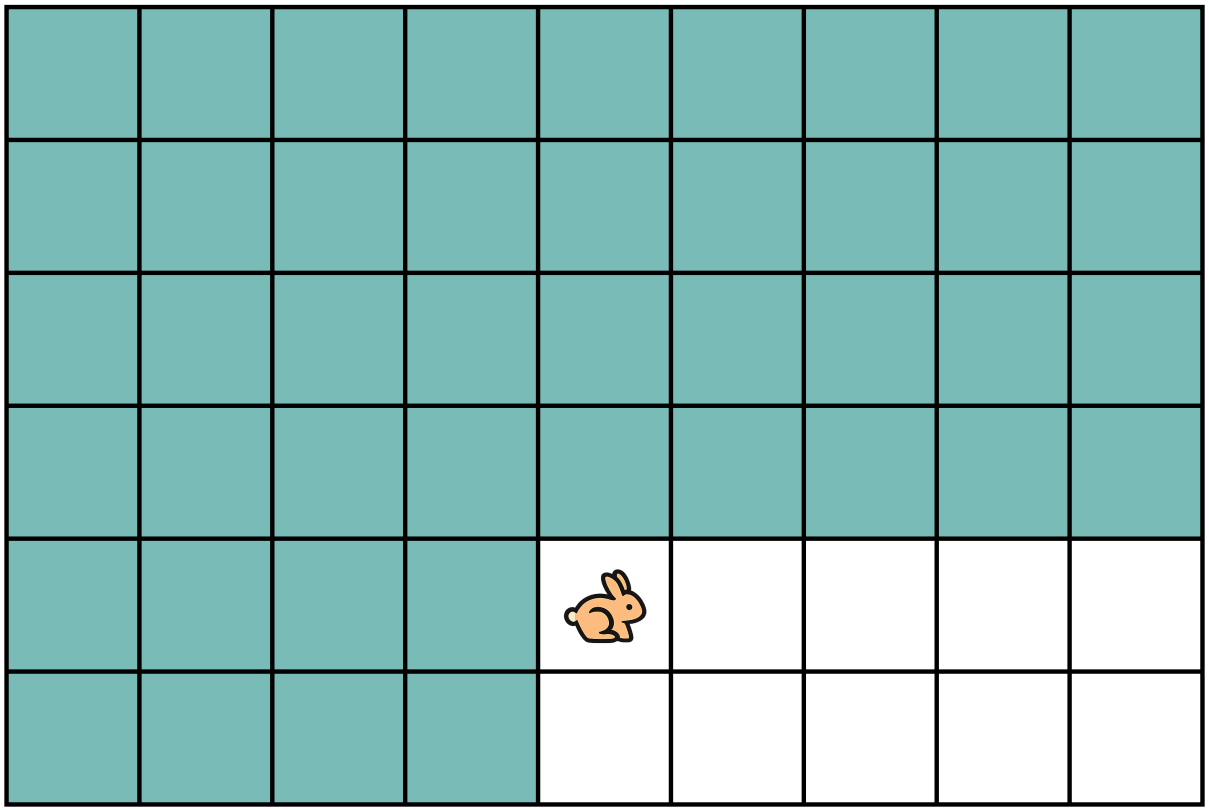
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



5×6

Maze problem

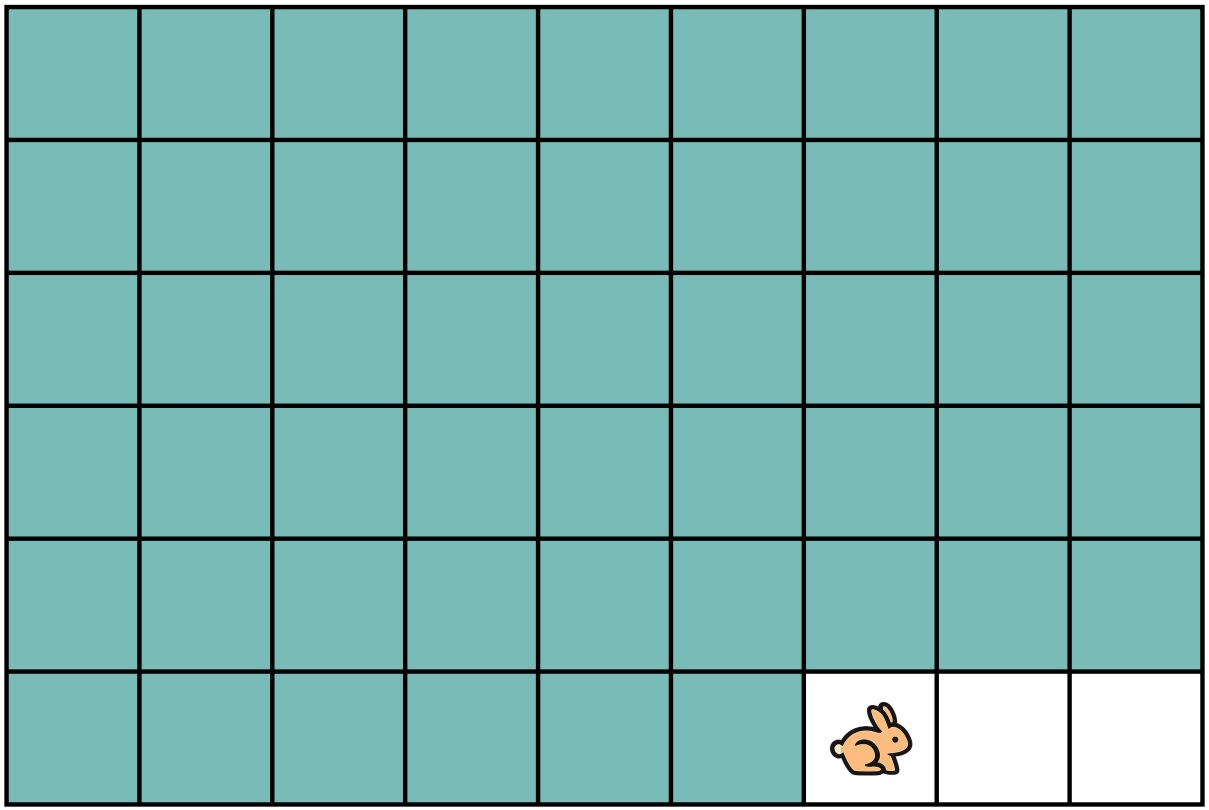
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



2×5

Maze problem

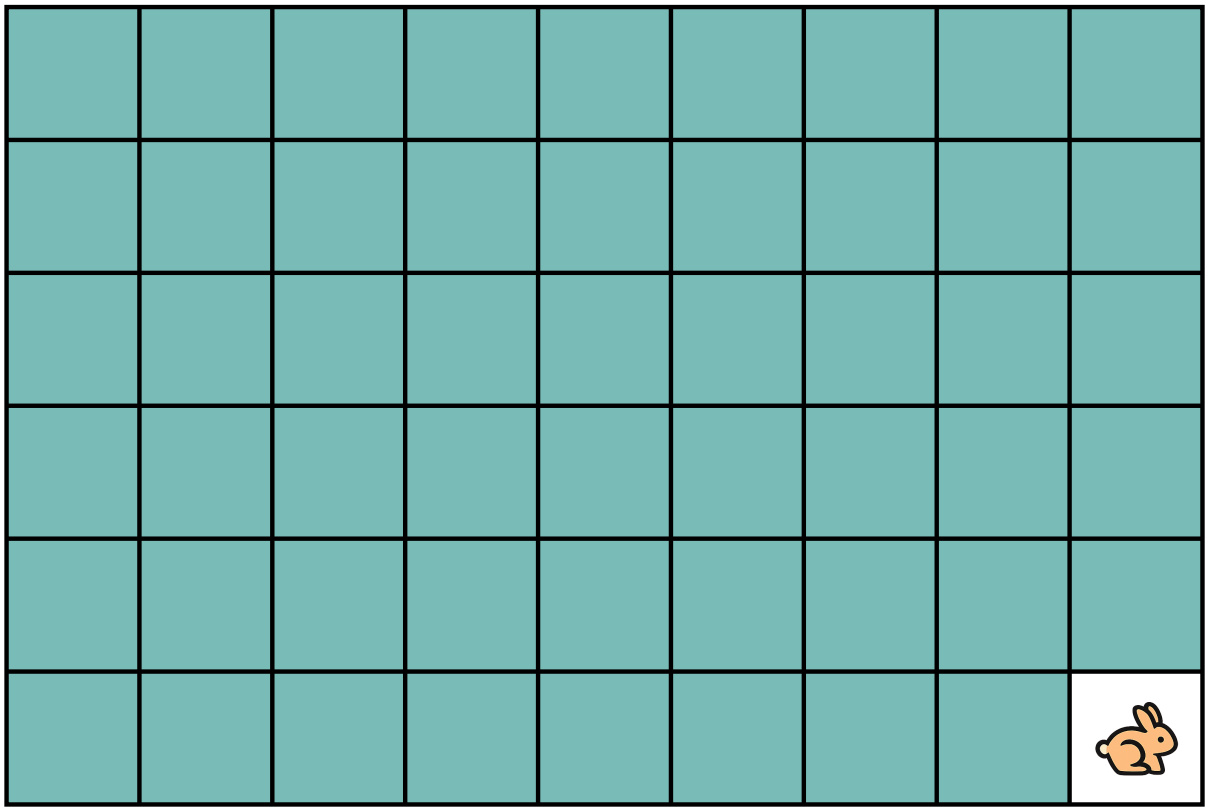
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



1×3

Maze problem

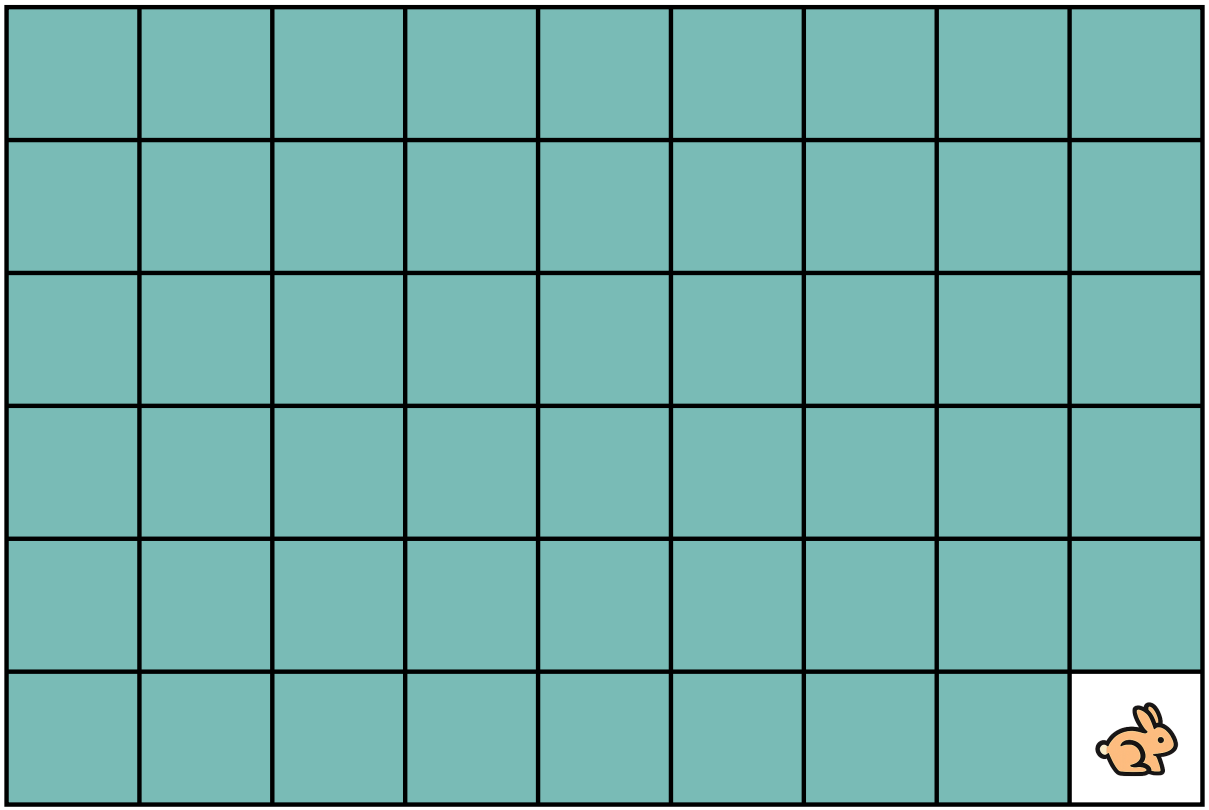
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



1×1

Maze problem - corner case

Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?

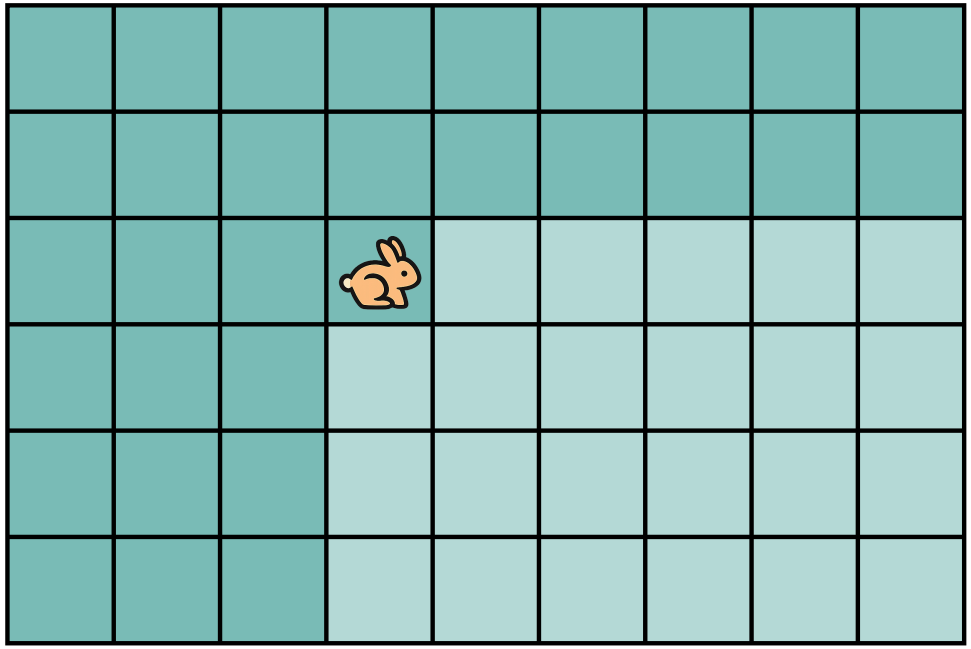


1×1

Rabbit has no more moves left!
It's s useful to think about corner cases.
Here rabbit can only reach the destination in exactly one way: do nothing.

Maze problem - let's construct a recursive solution

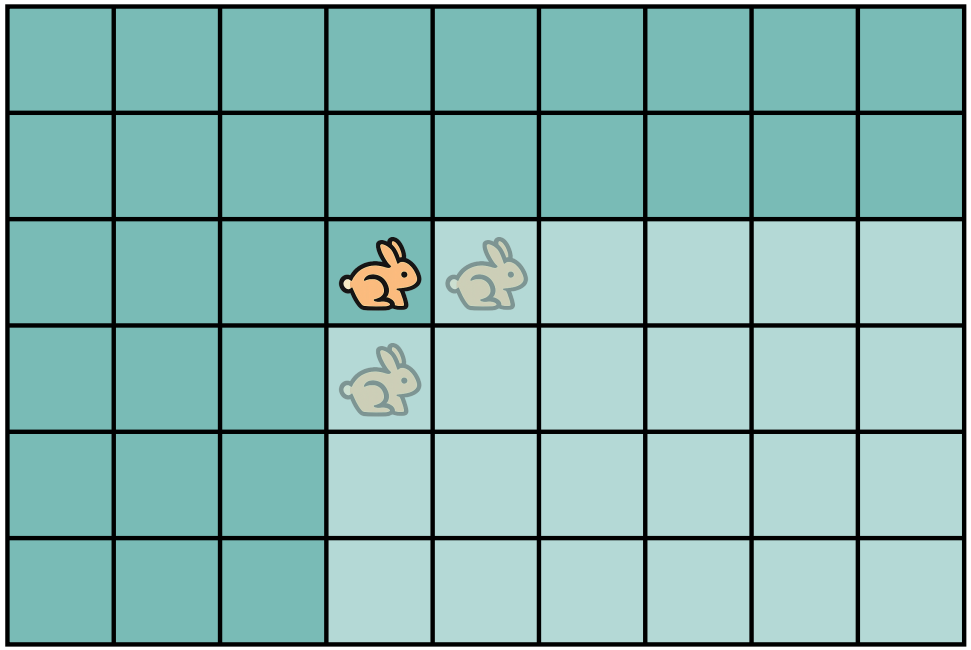
Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



4×6

Maze problem

Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?

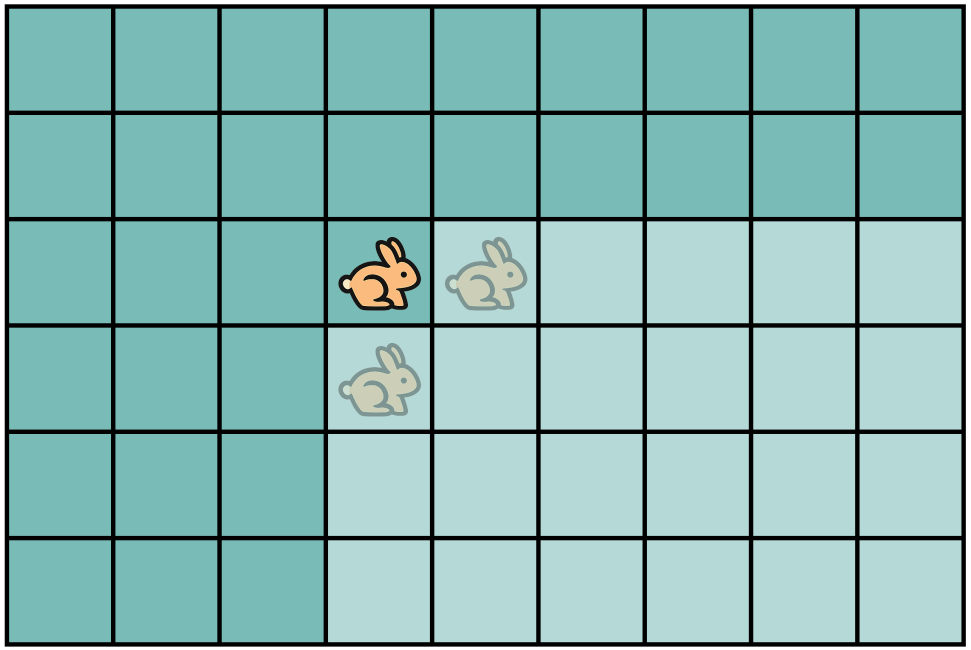


4×6

If rabbit moves right, it has to solve the subproblem of size 4×5.
If rabbit moves down, it has to solve the subproblem of size 3×6.
Since the rabbit can go either direction, we can take a sum of both subproblems.

Maze problem

Given an $N \times M$ grid, in how many ways can a rabbit get from the top-left to the bottom-right corner if it can only move down or right?



4×6

4×5

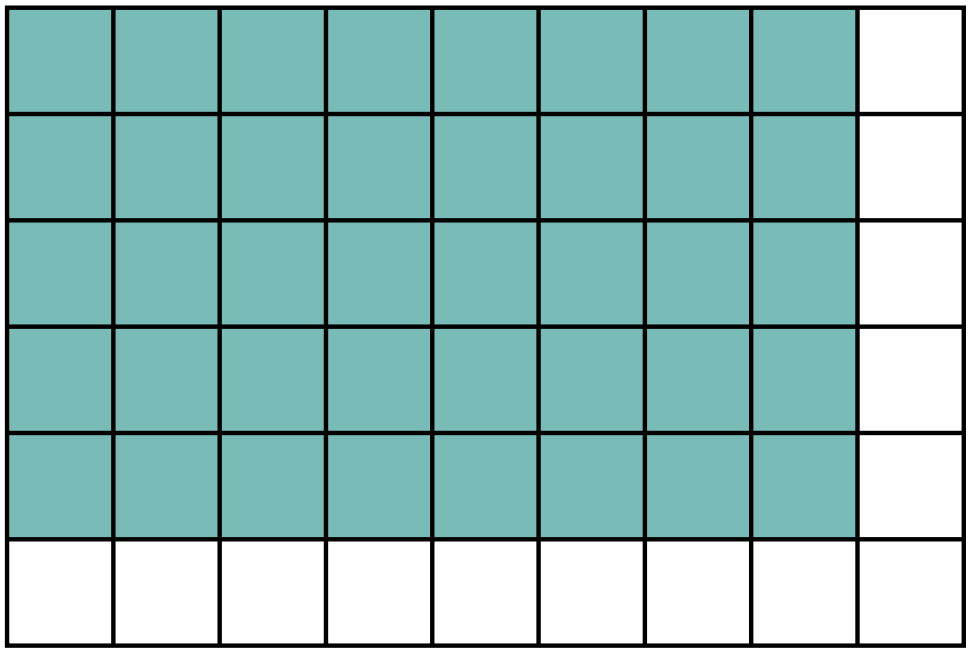
3×6

$$solution(1, 1) = 1$$

$$solution(n, m) = solution(n - 1, m) + solution(n, m - 1)$$

Maze problem

But... what about the boundary cases? What if rabbit is in the bottom row or in the last right column?



Then, we can assume that there is only way to reach bottom right corner once rabbit gets to the last row or column.

Maze problem

```
def grid_paths(n, m):
    memo = {}

    # Base cases
    for i in range(1, n + 1):
        memo[(i, 1)] = 1
    for j in range(1, m + 1):
        memo[(1, j)] = 1

    # Recursive formula
    for i in range(2, n + 1):
        for j in range(2, m + 1):
            memo[(i, j)] = memo[(i - 1, j)] + memo[(i, j - 1)]

    return memo[(n, m)]
```

Maze problem

```
print(grid_paths(18,6))
def grid_paths(n, m):
    memo = {}

    # Base cases
    for i in range(1, n + 1):
        memo[(i, 1)] = 1
    for j in range(1, m + 1):
        memo[(1, j)] = 1

    # Recursive formula
    for i in range(2, n + 1):
        for j in range(2, m + 1):
            memo[(i, j)] = memo[(i - 1, j)] + memo[(i, j - 1)]

    return memo[(n, m)]
```

Maze problem

```
def grid_paths(n, m):
    memo = {}

    # Base cases
    for i in range(1, n + 1):
        memo[(i, 1)] = 1
    for j in range(1, m + 1):
        memo[(1, j)] = 1

    # Recursive formula
    for i in range(2, n + 1):
        for j in range(2, m + 1):
            memo[(i, j)] = memo[(i - 1, j)] + memo[(i, j - 1)]

    return memo[(n, m)]
```

```
print(grid_paths(18,6))
```

Output: 26334

Maze problem

```
def grid_paths(n, m):
    memo = {}

    # Base cases
    for i in range(1, n + 1):
        memo[(i, 1)] = 1
    for j in range(1, m + 1):
        memo[(1, j)] = 1

    # Recursive formula
    for i in range(2, n + 1):
        for j in range(2, m + 1):
            memo[(i, j)] = memo[(i - 1, j)] + memo[(i, j - 1)]

    return memo[(n, m)]
```

```
print(grid_paths(18,6))
```

Output: 26334

```
print(grid_paths(75,19))
```

Maze problem

```
def grid_paths(n, m):
    memo = {}

    # Base cases
    for i in range(1, n + 1):
        memo[(i, 1)] = 1
    for j in range(1, m + 1):
        memo[(1, j)] = 1

    # Recursive formula
    for i in range(2, n + 1):
        for j in range(2, m + 1):
            memo[(i, j)] = memo[(i - 1, j)] + memo[(i, j - 1)]

    return memo[(n, m)]
```

```
print(grid_paths(18,6))
```

Output: 26334

```
print(grid_paths(75,19))
```

Output:

587318294164316715 0

Key Takeaways

1) Define the subproblem

2) What's the base case?

3) Find recursive formula for general solution

Assignment A3: Optimization by DP

Go to:

https://github.com/TBIAPBC/APBC2026/tree/master/A3

Happy hacking!